

Implementación y Verificación de un Clúster de Base de Datos Distribuida con CockroachDB aplicado al PFC

GA-SUM-03 / PE-U3 – Clúster BDD aplicado al PFC ACC – Soporte Técnico ISP

Código de equipo	D
PFC de referencia	ACC — Soporte Técnico ISP
Título del sistema	Sistema de Gestión de Solicitudes para Soporte Técnico de Internet

Integrante	Rol	PFC de origen
Aucatoma Celorio, Jhinson Stalyn	Responsable de documentación	AGLS – TiendaTech
Alvarez Parraga, Jeremy Alexis	Responsable de calidad	ACC – Soporte Técnico ISP
Carpio Mendoza, Carlos José	Líder de desarrollo	ACC – Soporte Técnico ISP

Justificación de la elección del PFC (máximo 100 palabras). Se eligió el dominio ACC porque el ticket de soporte tiene un dato muy natural para dividir la información: la zona geográfica del cliente (centro, norte o sur de Quevedo). En la vida real, un técnico casi siempre revisa solo los tickets de su propia zona, así que dividir la tabla por esa columna no es un ejercicio forzado, sino algo que tendría sentido usar en producción. Además, la cantidad de tickets que se registran y actualizan constantemente permite poner a prueba de forma clara tanto la tolerancia a fallos como el rendimiento del clúster distribuido.

Resumen

Este informe documenta la implementación, verificación y evaluación de un clúster de base de datos distribuida de tres nodos con CockroachDB, aplicado al esquema de datos del sistema de gestión de solicitudes de soporte técnico de Internet (PFC ACC). Se desarrolla el fundamento teórico de bases de datos distribuidas exigido por la guía (arquitecturas, fragmentación y replicación, control de concurrencia y consenso, y el espectro de consistencia, disponibilidad y PACELC), se describe la configuración del clúster en contenedores Docker y la estrategia de fragmentación horizontal de la tabla `tickets` por zona geográfica de atención (centro, norte y sur de Quevedo), se verifica experimentalmente la tolerancia a fallos mediante el algoritmo de consenso Raft, y se presenta un análisis comparativo cuantitativo de rendimiento entre el clúster distribuido y una instancia de nodo único sobre cinco consultas representativas del dominio. Los resultados muestran que la ventaja del clúster distribuido no es uniforme: resulta favorable en consultas con carga de cómputo significativa (agregaciones, *joins*), mientras que el nodo único es más rápido en lecturas puntuales por clave primaria, donde el costo fijo de coordinación distribuida no tiene contrapartida en paralelismo real.

Abstract

This report documents the implementation, verification, and evaluation of a three-node distributed database cluster built with CockroachDB, applied to the data schema of a ticket management system for Internet technical support (PFC ACC). The theoretical foundation of

distributed databases required by the practice guide is developed, covering distributed architectures, fragmentation and replication, distributed concurrency control and consensus, and the consistency–availability–PACELC spectrum. The cluster configuration in Docker containers and the horizontal fragmentation strategy applied to the `tickets` table by geographic service zone (Quevedo’s centro, norte, and sur) are described, fault tolerance is verified experimentally through the Raft consensus algorithm, and a quantitative comparative performance analysis is presented between the distributed cluster and a single-node instance across five representative queries of the domain. The results show that the advantage of the distributed cluster is not uniform: it is favorable for queries with significant computational load (aggregations, joins), whereas the single node is faster for point reads by primary key, where the fixed cost of distributed coordination has no counterpart in real parallelism.

1. Introducción

Las bases de datos distribuidas (BDD) han pasado de ser una alternativa de nicho a convertirse en el estándar de facto para sistemas que exigen alta disponibilidad, escalabilidad horizontal y tolerancia a fallos geográficos [1], [2]. CockroachDB es un representante moderno de la familia *NewSQL*: combina la interfaz y las garantías transaccionales de una base de datos relacional tradicional con la arquitectura de fragmentación y replicación automática propia de los sistemas NoSQL de gran escala, apoyándose en el algoritmo de consenso Raft [3] para coordinar las réplicas de cada fragmento de datos [4].

Este trabajo aplica estos conceptos al Proyecto Fin de Curso (PFC) del equipo ACC (Soporte Técnico ISP), cuyo núcleo funcional –materializado en el bounded context de Gestión de Tickets y su microservicio `ticket-service`– es la gestión del ciclo de vida de solicitudes de soporte técnico de Internet, distribuidas en tres zonas operativas de la ciudad de Quevedo. El objetivo de esta práctica es doble: (a) demostrar experimentalmente que la fragmentación horizontal y la replicación configuradas se comportan según la teoría –en particular, que el sistema sobrevive a la caída de un nodo sin pérdida de disponibilidad ni de datos– y (b) cuantificar el costo/beneficio real de la distribución frente a una instancia de nodo único, en vez de asumirlo por argumento de autoridad.

A diferencia del resto de la arquitectura de microservicios del PFC ACC –donde cada bounded context (Autenticación, Tickets, Notificaciones, Inteligencia Artificial, Reportes) posee su propia base de datos siguiendo el patrón *database-per-service*– esta práctica no introduce un sexto motor de base de datos independiente, sino que verifica cómo se comportaría el esquema del microservicio `ticket-service` si, en lugar de una única instancia PostgreSQL, se desplegara sobre un clúster distribuido de tres nodos. Esto permite ejercitar sobre datos y consultas reales del dominio los conceptos de fragmentación, replicación y consenso que la sola arquitectura de microservicios –con bases de datos independientes pero cada una internamente centralizada– no pone a prueba por sí sola.

El resto del documento se organiza así: la Sección 2 desarrolla el fundamento teórico exigido por la guía; la Sección 3 documenta el procedimiento aplicado con evidencia reproducible; la Sección 4 presenta los resultados experimentales y el análisis comparativo de rendimiento; la Sección 6 discute críticamente los hallazgos y sus limitaciones metodológicas; y la Sección 7 cierra con las conclusiones.

2. Fundamento teórico

2.1. Fundamentos y arquitecturas de bases de datos distribuidas

Una base de datos distribuida (BDD) es una colección de datos lógicamente interrelacionados que se encuentran físicamente dispersos en distintos sitios de una red de computadoras, de modo

que cada sitio cuenta con un grado de autonomía de procesamiento y, sin embargo, coopera con los demás para dar al usuario la impresión de estar interactuando con un único sistema centralizado [2], [5]. El sistema de gestión de bases de datos distribuidas (SGBDD) es el software que administra esa colección de datos y provee las facilidades de acceso necesarias para que dicha distribución sea, en la mayor medida posible, transparente al usuario [2], [5]. Esta definición es relevante para el PFC ACC – Soporte Técnico ISP, dado que la práctica exige llevar el esquema de datos del microservicio `ticket-service` (tablas como `tickets` y `technicians`) a un clúster de tres nodos, es decir, transformar una base de datos originalmente centralizada en una BDD en el sentido estricto del término.

Date propuso, a partir del principio fundamental de que “un sistema distribuido debería comportarse, desde la perspectiva del usuario, exactamente igual que un sistema no distribuido”, un conjunto de doce objetivos que sirven como marco de referencia para caracterizar cualquier SGBDD [5]. Estos objetivos son: (1) autonomía local, (2) no dependencia de un sitio central, (3) operación continua, (4) independencia de ubicación, (5) independencia de fragmentación, (6) independencia de replicación, (7) procesamiento de consultas distribuidas, (8) administración de transacciones distribuidas, (9) independencia de hardware, (10) independencia de sistema operativo, (11) independencia de red y (12) independencia de SGBD [2], [5]. Özsü y Valdúriez retoman este mismo marco al describir la transparencia de distribución como el objetivo central de todo SGBDD, y precisan que ninguno de estos doce objetivos se alcanza jamás de forma absoluta en un sistema real, sino que cada implementación los satisface en distinto grado [2]. Para el clúster de CockroachDB de la práctica, los objetivos más exigidos son la independencia de fragmentación (objetivo 5) y la administración de transacciones distribuidas (objetivo 8), ya que ambos se verifican explícitamente al ejecutar `SHOW RANGES` sobre la tabla `tickets` y al comprobar la continuidad del servicio ante la caída de un nodo.

En cuanto a las arquitecturas de referencia, la literatura distingue al menos tres modelos. La arquitectura *cliente-servidor* separa con claridad las responsabilidades: los servidores administran los datos y procesan las consultas, mientras que los clientes proveen la interfaz y la lógica de aplicación, de forma similar a como un sistema centralizado atendido por múltiples clientes remotos [2], [6]. La arquitectura *colaborativa* (también llamada de igual a igual o *peer-to-peer*) elimina esa distinción funcional: cada sitio ejecuta una instancia completa del SGBDD y coopera con los demás en pie de igualdad para resolver una consulta global, que es exactamente el modelo que sigue un clúster de CockroachDB, en el que cualquier nodo puede recibir una petición y coordinar su ejecución [2], [6]. Finalmente, la arquitectura de *middleware* aparece cuando los sitios integrantes ejecutan SGBD distintos y autónomos (un sistema multibase de datos o federado); en este caso se introduce una capa de mediadores y envoltorios (*mediator/wrapper*) que traduce un esquema conceptual global a los esquemas locales heterogéneos de cada fuente [2], [7].

Un SGBDD debe además ofrecer distintas formas de transparencia, entendidas como la capacidad del sistema de ocultar al usuario los detalles de implementación de la distribución. La *transparencia de fragmentación* permite formular consultas sobre las relaciones completas sin conocer cómo están particionadas físicamente [1], [2]. La *transparencia de replicación* permite operar como si existiera una única copia de cada dato, aun cuando existan varias réplicas [1], [2]. La *transparencia de ubicación* evita que el usuario deba conocer el sitio físico donde reside cada fragmento o réplica [2], [6]. La *transparencia de acceso* permite invocar operaciones locales y remotas con la misma interfaz [6], [8]. La *transparencia de concurrencia* garantiza que la ejecución simultánea de varias transacciones no altere el resultado que cada una percibe, como si se ejecutara en solitario [2], [6]. La *transparencia de fallos* oculta, en la medida de lo posible, la caída de un nodo o de un enlace de comunicación [6], [8]. La *transparencia de escalabilidad* permite incorporar nuevos nodos sin que ello obligue a rediseñar las aplicaciones existentes [6], [8]. Por último, la *transparencia de evolución* (una extensión distribuida de la independencia lógica de datos) permite modificar el esquema global o la codificación de los registros a lo largo del tiempo sin romper la compatibilidad con las aplicaciones cliente ya desplegadas [1], [2].

Estas ocho transparencias se verificarán de forma práctica en la Sección 3 al comprobar que las consultas SQL sobre la tabla principal del PFC no requieren modificación alguna tras aplicar la fragmentación ni durante la caída de un nodo del clúster.

Finalmente, es necesario distinguir entre BDD homogéneas y heterogéneas. Un sistema homogéneo utiliza el mismo motor de SGBD, el mismo modelo de datos y, en general, el mismo lenguaje de consulta en todos sus sitios, lo que simplifica de forma considerable el procesamiento de consultas distribuidas y la administración de transacciones [2], [5]. Un sistema heterogéneo, en cambio, integra motores distintos y autónomos (por ejemplo, un sistema relacional junto con un almacén documental), lo que exige mecanismos adicionales de traducción de esquemas y de resolución de conflictos semánticos entre fuentes [2], [7]. El clúster de tres nodos CockroachDB que se implementa en esta práctica corresponde a una BDD homogénea, ya que los tres nodos ejecutan el mismo motor y comparten un único esquema lógico para las tablas del PFC ACC – Soporte Técnico ISP, lo cual favorece precisamente el nivel de transparencia que exige el objetivo de independencia de fragmentación de Date.

2.2. Fragmentación y replicación de datos

La fragmentación es el proceso de diseño de la distribución mediante el cual una relación global R se descompone en subconjuntos denominados *fragmentos*, que se asignan físicamente a los distintos nodos del sistema con el propósito de acercar los datos a las aplicaciones que los consumen, reducir los costos de transmisión y habilitar el paralelismo en la ejecución de consultas [2], [9], [10]. Para que un esquema de fragmentación sea correcto debe satisfacer tres condiciones formales: *completitud*, que exige que todo elemento de datos de R se encuentre en al menos un fragmento; *reconstrucción*, que garantiza la existencia de un operador relacional capaz de recomponer R a partir de sus fragmentos sin pérdida de información; y *disjunción*, exclusiva de la fragmentación horizontal, que impide que una misma tupla pertenezca simultáneamente a dos fragmentos, evitando redundancia no controlada en el nivel de diseño [2], [9], [11]. Este mismo principio de particionar los datos para acercarlos a su patrón de acceso es el que subyace, en el ecosistema de bases de datos distribuidas modernas, al particionamiento por rango o por hash que describe Kleppmann como mecanismo central de escalabilidad horizontal [1].

La fragmentación horizontal primaria de una relación R mediante un predicado de selección p_i se define como:

$$R_i = \sigma_{p_i}(R) \quad (1)$$

cumpléndose las siguientes condiciones [9], [10]:

$$\bigcup_{i=1}^n R_i = R \quad (\text{completitud}) \quad (2)$$

$$R_i \cap R_j = \emptyset \quad \forall i \neq j \quad (\text{disjunción}) \quad (3)$$

donde los predicados p_i se construyen como predicados *minterm* –conjunciones de predicados simples o de sus negaciones– lo que garantiza por construcción la disjunción de los fragmentos resultantes.

Por su parte, la fragmentación horizontal derivada particiona una relación miembro R_2 en función de la fragmentación previamente aplicada a su relación propietaria R_1 , con la que mantiene un vínculo de clave foránea, mediante el operador de semi-junta $\ltimes$ [10], [11]:

$$R_{2i} = R_2 \ltimes R_{1i} = R_2 \ltimes \sigma_{p_i}(R_1) \quad (4)$$

Esta estrategia co-localiza las tuplas hijas junto a sus tuplas padre, preservando la localidad de las operaciones JOIN y minimizando la transferencia de datos entre nodos durante la evaluación de consultas distribuidas. Para el dominio del PFC de referencia (ACC – Soporte Técnico ISP), la fragmentación horizontal primaria se aplica sobre la tabla `tickets`, empleando como predicado

p_i la zona geográfica de atención del cliente (QUEVEDO_CENTRO, QUEVEDO_NORTE, QUEVEDO_SUR), mientras que una tabla de eventos asociada, como el historial de cambios de estado del ticket, se fragmentaría de forma derivada respecto a los fragmentos ya definidos de `tickets`, preservando la localidad de las consultas de tipo JOIN entre ambas tablas.

La fragmentación vertical, en contraste, divide una relación por sus atributos en lugar de por sus tuplas, mediante el operador de proyección π , generando fragmentos:

$$R_i = \pi_{K, A_i}(R) \quad (5)$$

donde K es la clave primaria –replicada en todos los fragmentos para posibilitar la reconstrucción mediante junta natural– y A_i un subconjunto de atributos no clave, siendo la reconstrucción:

$$R = R_1 \bowtie R_2 \bowtie \dots \bowtie R_m \quad (6)$$

La decisión de qué atributos agrupar en cada fragmento se sustenta en el análisis de *afinidad de atributos*: a partir de la matriz de uso, que registra qué consultas acceden a qué atributos y con qué frecuencia, se construye la matriz de afinidad AA , cuyo elemento $aff(A_j, A_k)$ cuantifica la frecuencia con que dos atributos son accedidos conjuntamente por las mismas transacciones; sobre esta matriz operan algoritmos de agrupamiento como el *Bond Energy Algorithm* (BEA) y sus variantes optimizadas, que reordenan filas y columnas para maximizar la medida de afinidad global y determinar el punto óptimo de partición, agrupando en un mismo fragmento los atributos de alta afinidad mutua y reduciendo el número de juntas distribuidas que exige la carga de trabajo [9], [10].

La fragmentación mixta o híbrida combina de manera anidada ambas estrategias, aplicando primero una división vertical y, sobre cada fragmento resultante, una horizontal (o en el orden inverso):

$$R_{ij} = \sigma_{p_j}(\pi_{K, A_i}(R)) \quad (7)$$

efectuándose la reconstrucción al componer los operadores inversos ($\bowtie$ y $\cup$) en el orden contrario al de la descomposición [9], [11]. Esta estrategia resulta apropiada cuando coexisten patrones de acceso heterogéneos sobre una misma relación, y los enfoques recientes de particionamiento dinámico demuestran que la combinación de criterios verticales y horizontales, ajustada a la evolución de la carga de trabajo, mejora simultáneamente el desempeño de las consultas y otros objetivos de diseño [9], [10].

Finalmente, la estrategia de replicación determina cómo se propagan las actualizaciones entre las réplicas de un fragmento, manteniendo copias en múltiples nodos para elevar la disponibilidad, la tolerancia a fallos y el desempeño de lectura [12], [13]. La *replicación síncrona* garantiza que una transacción no se confirme hasta que un conjunto suficiente de réplicas –típicamente un quórum mayoritario– reconozca la escritura, ofreciendo consistencia fuerte a costa de mayor latencia, pues requiere rondas adicionales de comunicación y coordinación entre nodos, y de menor disponibilidad ante particiones de red; la *replicación asíncrona*, en cambio, confirma la escritura en el nodo líder y propaga los cambios posteriormente en segundo plano, reduciendo la latencia percibida por el cliente y elevando la disponibilidad, pero introduciendo una ventana de inconsistencia temporal en la que réplicas seguidoras pueden servir datos obsoletos e incluso perder escrituras confirmadas si el líder falla antes de propagarlas [1], [12]. Esta ventana de inconsistencia temporal corresponde, dentro del espectro formal de modelos de consistencia, a una garantía más débil que la consistencia fuerte y más cercana a la consistencia eventual, según la clasificación de Viotti y Vukolić [14]. Este compromiso entre consistencia, latencia y disponibilidad se deriva directamente de los teoremas CAP y PACELC: bajo partición debe elegirse entre disponibilidad y consistencia y, en ausencia de partición, entre latencia y consistencia [13], [15]. CockroachDB adopta un esquema de replicación síncrona basado en consenso Raft, en el que una escritura solo se considera confirmada tras ser replicada en la mayoría del quórum de réplicas del rango correspondiente, decisión de diseño que privilegia la consistencia serializable sobre la latencia mínima y que será verificada experimentalmente en la Sección 3.3 mediante la prueba de tolerancia a fallos [3], [4].

2.3. Control de concurrencia distribuido y consenso

El problema de la concurrencia en una BDD surge porque varias transacciones pueden ejecutarse de forma simultánea sobre fragmentos y réplicas ubicados en distintos nodos, de modo que, además de las anomalías clásicas de un sistema centralizado (lecturas sucias, actualizaciones perdidas, lecturas no repetibles), aparecen problemas propios del entorno distribuido: la falta de un reloj global compartido dificulta establecer un orden único entre operaciones, y la necesidad de coordinar múltiples copias de un mismo dato introduce un costo de comunicación que no existe en un SGBD centralizado [2], [8], [16]. Mantener la propiedad de aislamiento en este escenario exige que cada transacción se comporte como si fuera la única en ejecución sobre el sistema completo, sin que ello implique serializar por completo el acceso a los datos, ya que esto eliminaría el beneficio de la distribución [8], [16].

El mecanismo más extendido para resolver este problema es el bloqueo en dos fases (*Two-Phase Locking*, 2PL) distribuido, en el cual cada transacción atraviesa una fase de crecimiento –durante la cual solo puede adquirir bloqueos– seguida de una fase de liberación –durante la cual solo puede liberarlos–, garantizándose con ello la serializabilidad del historial de ejecución aun cuando los bloqueos se administren en sitios distintos [2], [16]. La variante estricta (*Strict Two-Phase Locking*, S2PL) exige además que todos los bloqueos exclusivos se mantengan hasta el instante mismo del *commit* o del *abort* de la transacción, lo que evita que otras transacciones lean datos escritos por una transacción que aún podría deshacerse, a costa de una menor concurrencia efectiva [2], [8]. En un clúster distribuido, aplicar S2PL implica coordinar los bloqueos de todos los nodos que participan en una transacción, lo cual introduce la posibilidad de bloqueos mutuos entre sitios (*deadlocks* distribuidos) que no pueden detectarse observando un único nodo de forma aislada.

La detección de estos bloqueos mutuos distribuidos se apoya típicamente en un grafo de espera (*wait-for graph*, WFG), en el que cada nodo representa una transacción y cada arco dirigido $T_i \rightarrow T_j$ indica que T_i espera un recurso retenido por T_j ; la existencia de un bloqueo mutuo se corresponde con la existencia de un ciclo en este grafo [2], [17]. En un entorno distribuido, ningún sitio posee por sí solo el grafo de espera completo, por lo que se emplean esquemas centralizados (un único nodo consolida los WFG locales de todos los demás), jerárquicos o completamente distribuidos, cada uno con compromisos distintos entre el tráfico de mensajes generado, la probabilidad de detectar bloqueos ficticios (*phantom deadlocks*) y el tiempo que tarda el sistema en reconocer un ciclo real; el estudio de Bukhres y Magel muestra experimentalmente que el esquema parcialmente distribuido ofrece, en general, el mejor equilibrio entre estos factores [8], [17].

Además del control de concurrencia, una transacción distribuida debe garantizar atomicidad: o se confirma en todos los nodos participantes o se deshace en todos ellos. El protocolo de *commit* en dos fases (2PC) es el mecanismo clásico para lograrlo: en una primera fase, el coordinador solicita a cada participante su disposición a confirmar la transacción (*votar*), y en una segunda fase envía la decisión final (confirmar o deshacer) una vez recibidos todos los votos [2], [18]. El problema principal de 2PC es que, si el coordinador falla después de recolectar los votos pero antes de difundir la decisión final, los participantes que ya votaron a favor quedan bloqueados indefinidamente en un estado incierto, reteniendo sus recursos hasta que el coordinador se recupere [2], [19]. Este escenario de bloqueo, ampliamente documentado, es precisamente el que motivó el desarrollo de protocolos de *commit* no bloqueantes: Skeen demostró las condiciones necesarias y suficientes que un protocolo de terminación debe cumplir para evitar el bloqueo ante la falla de un coordinador [19]. En el contexto actual de microservicios, 2PC sigue empleándose para coordinar transacciones que abarcan varios servicios, aunque su naturaleza bloqueante limita su escalabilidad bajo alta concurrencia y contención [18].

El protocolo de *commit* en tres fases (3PC) introduce una fase intermedia de *pre-commit* entre la votación y la confirmación definitiva, de modo que, si el coordinador falla, los participantes pueden determinar de forma autónoma el estado global de la transacción consultando entre sí, sin

quedar bloqueados a la espera de la recuperación del coordinador [19], [20]. Esta fase adicional reduce el riesgo de bloqueo indefinido observado en 2PC, aunque incrementa el número de rondas de mensajes requeridas para completar cada transacción [20].

Finalmente, el algoritmo de consenso Raft, empleado por CockroachDB para replicar cada rango de datos entre los nodos del clúster, resuelve un problema más general que el de la simple atomicidad de una transacción: lograr que un conjunto de nodos coincida en una misma secuencia de operaciones (el *log replicado*) aun cuando algunos de ellos fallen o la red se particione temporalmente [3], [4]. Raft descompone el consenso en tres subproblemas: la elección de líder, mediante la cual los nodos seleccionan por votación mayoritaria a un único líder para un período determinado (*term*); la replicación del log, en la que el líder recibe las operaciones de los clientes, las agrega a su log local y las reenvía a los demás nodos, considerando una entrada confirmada solo cuando la ha recibido la mayoría del clúster; y la seguridad del protocolo, que garantiza mediante la propiedad de *completitud del líder* que ningún nodo pueda convertirse en líder si no contiene todas las entradas ya confirmadas por líderes anteriores, evitando así que una entrada confirmada se pierda [3], [4]. Esta última garantía es la que se verificará de forma empírica en la prueba de tolerancia a fallos de la Sección 3.3: al detener uno de los tres nodos del clúster, los dos nodos restantes conservan la mayoría del quórum y pueden, por lo tanto, continuar aceptando escrituras sin perder ninguna operación ya confirmada [3], [4].

2.4. Consistencia, disponibilidad, PACELC y sistemas NewSQL/NoSQL

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura original de Brewer, establece que, ante una partición de red, un sistema distribuido no puede garantizar simultáneamente consistencia (C) y disponibilidad (A): debe sacrificar una de las dos mientras la partición persiste [21], [22]. Formalmente, si P denota la ocurrencia de una partición de red, el teorema establece que no es posible satisfacer de forma simultánea:

$$C \wedge A \wedge P \quad (8)$$

es decir, un sistema real debe elegir entre las combinaciones CP (consistente pero potencialmente no disponible durante la partición) o AP (disponible pero potencialmente inconsistente durante la partición) [21], [22]. Brewer revisó una década después su propia conjetura y aclaró que CAP describe únicamente el comportamiento del sistema *durante* el breve intervalo en que persiste una partición, y que la mayoría de los sistemas reales no eligen una postura fija, sino que gestionan de forma dinámica el grado de consistencia sacrificado según la operación y el subconjunto de datos afectado, en lugar de renunciar por completo a la disponibilidad o a la consistencia de todo el sistema [21], [22].

Sin embargo, las particiones de red son eventos poco frecuentes en la mayoría de los despliegues, por lo que CAP resulta insuficiente para describir el comportamiento habitual de un sistema replicado en operación normal. Abadi propuso la formulación PACELC como extensión de CAP: ante una Partición (P), el sistema elige entre Availability (disponibilidad) y Consistencia; pero, en su ausencia (*Else*, E), el sistema elige entre Latencia (L) y Consistencia (C) [15], [23]. Esta segunda disyuntiva es, de hecho, la que determina el comportamiento cotidiano de un sistema geo-distribuido, ya que la mayor parte del tiempo de operación transcurre sin particiones activas: un esquema de replicación síncrona sacrifica latencia para preservar consistencia en todo momento, mientras que uno asíncrono prioriza la latencia a costa de una ventana de inconsistencia temporal [15], [23]. Para un sistema geo-distribuido, esta disyuntiva EL se vuelve más crítica cuanto mayor es la distancia física entre réplicas, ya que el tiempo de propagación de una confirmación síncrona crece con la latencia de red entre regiones.

Entre ambos extremos del espectro de consistencia definido por Viotti y Vukolić se ubican modelos intermedios: la consistencia secuencial exige un orden total acordado por todos los clientes, la consistencia causal preserva únicamente el orden entre operaciones causalmente relacionadas,

y la consistencia eventual –el extremo más débil– únicamente garantiza que, en ausencia de nuevas escrituras, todas las réplicas convergerán finalmente al mismo valor [1], [14]. CockroachDB se ubica en el extremo más fuerte de este espectro, ya que ofrece aislamiento serializable de forma nativa para todas las transacciones, incluso aquellas que abarcan varios rangos distribuidos entre distintos nodos: el sistema detecta y resuelve conflictos de escritura mediante un protocolo de concurrencia optimista basado en *timestamps* y reintentos, de modo que el resultado de cualquier conjunto de transacciones concurrentes es equivalente al de alguna ejecución serial de las mismas [2], [4]. Esta decisión de diseño es la que se sacrifica en latencia –según el eje EL de PACELC– para preservar la consistencia más fuerte del espectro.

Los sistemas NoSQL, en cambio, se ubican deliberadamente en un punto distinto del espectro para priorizar la disponibilidad y la latencia. Dynamo, el almacén clave-valor de Amazon, es representativo del modelo NoSQL de tipo clave-valor: prioriza explícitamente la disponibilidad mediante replicación sin líder por quórum y consistencia eventual con resolución de conflictos del lado del cliente [1], [24]. Cassandra, inspirado directamente en Dynamo, adopta un modelo columnar distribuido sin punto único de fallo, en el que el nivel de consistencia (*ONE*, *QUORUM*, *ALL*) puede ajustarse por operación, exponiendo explícitamente al desarrollador el compromiso entre consistencia y disponibilidad que Dynamo resuelve de forma más implícita [1], [25]. MongoDB, un sistema documental, ofrece un mecanismo de *consistencia ajustable* (*tunable consistency*) que permite seleccionar, por cada operación de lectura o escritura, el nivel de garantía deseado (por ejemplo, *majority write concern* o *linearizable read concern*), en lugar de fijar un único nivel para todo el sistema [1], [26]. En el extremo opuesto del espectro se encuentra Spanner, que ofrece consistencia externa apoyándose en TrueTime, un servicio de sincronización de relojes con incertidumbre acotada que le permite ordenar transacciones globalmente casi como si el sistema no estuviera distribuido [1], [27].

La Tabla 1 resume el análisis comparativo entre CockroachDB, MongoDB y Cassandra sobre el eje consistencia/disponibilidad/latencia (C/A/L), que sintetiza el compromiso PACELC particular que cada sistema prioriza.

Cuadro 1: Comparación de CockroachDB, MongoDB y Cassandra en el eje C/A/L

Criterio	CockroachDB	MongoDB	Cassandra
Consistencia por defecto	Serializable [4]	Ajustable, hasta linealizable [26]	Ajustable por operación [25]
Disponibilidad ante partición	Preserva C sobre A (CP)	Configurable según <i>write concern</i> [26]	Prioriza A sobre C (AP) [25]
Latencia típica de escritura	Mayor: consenso Raft por rango [4]	Media, según el <i>concern</i> elegido [26]	Menor: réplica sin líder fijo [25]
Compromiso PACELC	PC/EC (prioriza C)	Configurable según operación	PA/EL (prioriza A y L)

Nota: además de la fuente primaria citada en cada celda, el marco comparativo consistencia/disponibilidad/latencia se apoya de forma transversal en Kleppmann [1] y en el espectro de consistencia de Viotti y Vukolić [14].

Como muestra la Tabla 1, CockroachDB y Cassandra representan los dos extremos del espectro para el dominio de ACC – Soporte Técnico ISP: mientras que CockroachDB resulta apropiado cuando se requiere consistencia fuerte entre operaciones relacionadas –como la asignación atómica de un ticket a un técnico, donde dos técnicos no deben quedar asignados simultáneamente al mismo ticket–, Cassandra sería preferible únicamente si el sistema priorizara la disponibilidad de lectura por encima de la exactitud inmediata del estado del ticket, algo que no se ajusta al dominio transaccional de un sistema de gestión de solicitudes de soporte técnico. MongoDB, al ofrecer un espectro ajustable, permitiría migrar gradualmente entre ambos extremos según la colección de datos, aunque exige que el equipo de desarrollo decida de forma explícita, operación

por operación, qué nivel de consistencia aplicar.

3. Procedimiento aplicado

3.1. Levantamiento del clúster

El clúster se define en `docker-compose.yml` (Listado 1) con tres servicios (`crdb-node1/2/3`), cada uno con su propia `locality` de zona, un límite de memoria (`mem_limit: 512m`) y un `healthcheck` que consulta el endpoint interno `/health?ready=1` de CockroachDB.

Listing 1: Definición del clúster de 3 nodos (`docker-compose.yml`)

```
1 services:
2   crdb-node1:
3     image: cockroachdb/cockroach:latest-v23.2
4     container_name: crdb-node1
5     command: start --insecure --max-offset=4500ms
6               --locality=zone=quevedo-centro
7               --join=crdb-node1,crdb-node2,crdb-node3
8     ports:
9       - "26257:26257"
10      - "8080:8080"
11     mem_limit: 512m
12     healthcheck:
13       test: ["CMD", "curl", "-f", "http://localhost:8080/health?ready=1"]
14       interval: 10s
15       timeout: 5s
16       retries: 5
17       start_period: 20s
18     # crdb-node2 y crdb-node3 son analogos, con locality=quevedo-norte
19     # y locality=quevedo-sur respectivamente (ver docker-compose.yml completo)
```

Tras `docker compose up -d` e inicializar con `cockroach init -insecure`, la verificación con `cockroach node status -insecure` confirmó los tres nodos en `is_live=true` (Figura 1).

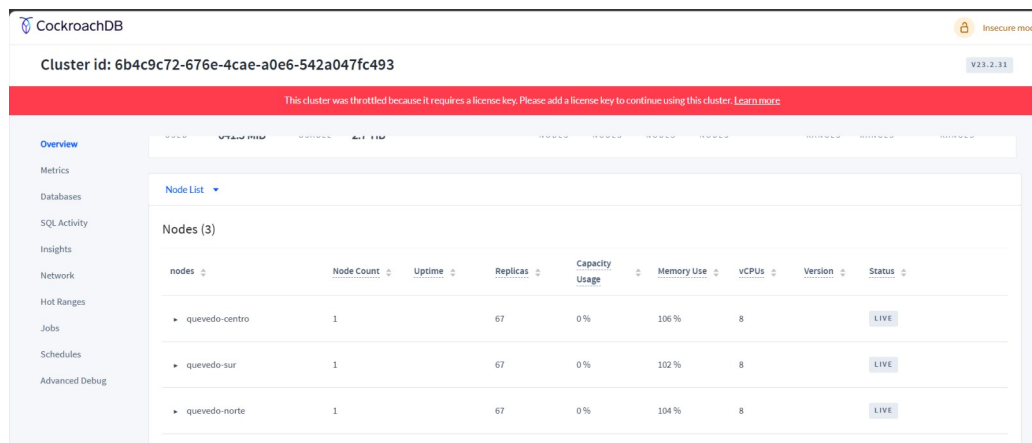


Figura 1: Dashboard de CockroachDB mostrando los 3 nodos activos (LIVE), 106+ rangos replicados y 0 rangos sub-replicados.

3.2. Esquema del PFC y fragmentación aplicada

El esquema (Listado 2) define `technicians` y `tickets`, esta última con clave primaria compuesta (`zone, id`) para que `zone` pueda usarse como clave de partición.

Listing 2: Fragmento de `sql/01_schema.sql`: tabla principal del dominio

```

1 CREATE TABLE tickets (
2     zone          STRING NOT NULL CHECK (zone IN
3         ('QUEVEDO_CENTRO','QUEVEDO_NORTE','QUEVEDO_SUR')),
4     id            UUID NOT NULL DEFAULT gen_random_uuid(),
5     client_id     UUID NOT NULL,
6     technician_id UUID REFERENCES technicians(id),
7     category      STRING,
8     priority      STRING,
9     status        STRING NOT NULL DEFAULT 'NUEVO',
10    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
11    sla_deadline   TIMESTAMPTZ,
12    resolved_at    TIMESTAMPTZ,
13    sla_breached   BOOL DEFAULT FALSE,
14    PRIMARY KEY (zone, id)
15 );

```

La fragmentación (Listado 3) se aplica en un paso independiente con `ALTER TABLE ... PARTITION BY LIST`, y cada partición se ancla preferentemente a su nodo de localidad manteniendo `num_replicas=3` para no sacrificar tolerancia a fallos.

Listing 3: `sql/02_partitions.sql`: fragmentación horizontal y política de zona

```

1 ALTER TABLE tickets PARTITION BY LIST (zone) (
2     PARTITION tickets_centro VALUES IN ('QUEVEDO_CENTRO'),
3     PARTITION tickets_norte  VALUES IN ('QUEVEDO_NORTE'),
4     PARTITION tickets_sur    VALUES IN ('QUEVEDO_SUR'),
5     PARTITION tickets_default VALUES IN (DEFAULT)
6 );
7
8 ALTER PARTITION tickets_centro OF TABLE tickets
9     CONFIGURE ZONE USING num_replicas = 3,
10     constraints = '{"+zone=quevedo-centro": 1}';
11 -- analogo para tickets_norte y tickets_sur

```

Con 20 005 filas cargadas mediante `scripts/seed_data.py` (distribución objetivo 40/30/30), la distribución real observada fue:

Cuadro 2: Distribución real de tickets por partición tras la carga

Zona	Filas	% del total
QUEVEDO_CENTRO	8 001	40.0 %
QUEVEDO_NORTE	6 003	30.0 %
QUEVEDO_SUR	6 001	30.0 %
Total	20 005	100 %

`SHOW RANGES FROM TABLE tickets` confirmó que cada partición reside en un rango separado, replicado en los tres nodos (`{1,2,3}`), lo que verifica empíricamente las tres condiciones de corrección de la Ecuación 2-3: cada fila pertenece a exactamente una partición (disjunción), la unión de las tres particiones más la partición `default` cubre el 100% de las filas (completitud), y `SELECT * FROM tickets` reconstruye la relación original sin duplicados ni omisiones (reconstrucción).

3.3. Prueba de tolerancia a fallos

Siguiendo el guion de la guía, se registró una consulta base, se detuvo `crdb-node2`, se repitió la consulta de inmediato, se esperó 30 segundos documentando `SHOW RANGES`, y finalmente se reinició el nodo midiendo el tiempo de reintegración. La Tabla 3 resume la evidencia recolectada (ver también el video en `evidencia/video_tolerancia.mp4`).

Cuadro 3: Cronología de la prueba de tolerancia a fallos

Paso	Resultado observado
Consulta base (3 nodos vivos)	<code>COUNT(*) = 20005</code> , ~1.5 ms
<code>docker stop crdb-node2</code>	Nodo detenido
Consulta inmediata tras la caída	<code>COUNT(*) = 20005</code> — responde correctamente por quórum (2 de 3 nodos)
<code>node status</code> con el nodo caído	<code>crdb-node2: is_live=false</code> ; los otros dos: <code>true</code>
<code>SHOW RANGES</code> tras 30 s	Réplicas siguen listadas en {1,2,3} — sin redistribución física; esperado, porque <code>server.time_until_store_dead</code> (5 min por defecto) aún no se cumple
<code>docker start crdb-node2</code>	Reinicio del nodo
Reintegración confirmada	<code>crdb-node2</code> vuelve a <code>is_live=true</code> en menos de ~65 s
Verificación de integridad	20 005 filas intactas, distribución 8001/6003/6001 sin cambios

La Figura 2 ilustra, como diagrama de secuencia de elaboración propia, el comportamiento interno de Raft durante este escenario: al perderse el líder o una réplica no líder, el grupo Raft del rango afectado sigue respondiendo mientras conserve el quórum, y solo se dispara una nueva elección de líder si el nodo caído era efectivamente el líder de ese rango.

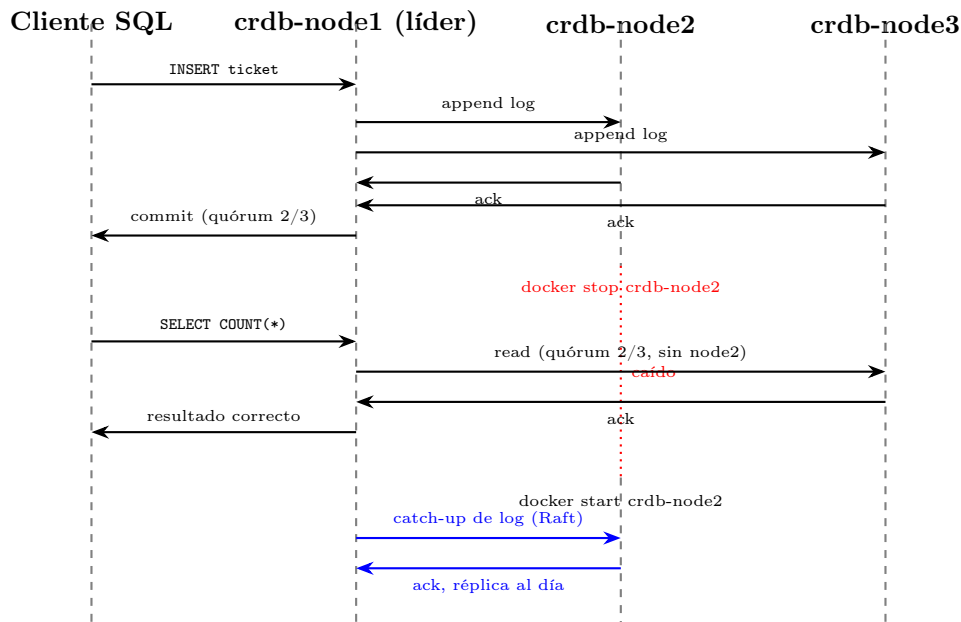


Figura 2: Diagrama de secuencia (elaboración propia) del comportamiento de Raft durante la caída y reintegración de `crdb-node2`: el quórum de 2/3 nodos mantiene la disponibilidad de lectura/escritura sin interrupción, y la reintegración se resuelve por *catch-up* de log, no por elección de líder (porque `crdb-node2` no era el líder del rango afectado).

4. Resultados y análisis comparativo de rendimiento

Las cinco consultas de `sql/03_queries.sql` se ejecutaron con `EXPLAIN ANALYZE` cinco veces en cada entorno (clúster de 3 nodos y nodo único), reportando la mediana para neutralizar outliers de arranque en frío y de contención de la VM de desarrollo (metodología completa en `evidencia/benchmark_summary.md`).

Cuadro 4: Comparativa de rendimiento: clúster (3 nodos) vs. nodo único

#	Consulta	Clúster (ms)	Nodo único (ms)	Factor*
Q1	Lectura por PK	4–5	2	0.4–0.5×
Q2	Rango por zona + fecha	7–26	8–11	0.3–1.6×
Q3	Agregación cruzando zonas	13–16	11–14	0.7–1.1×
Q4	<code>GROUP BY</code> zona+categoría (SLA)	26–29	34–66	1.3–2.3×
Q5	<code>JOIN</code> tickets↔técnicos	28–30	30–61	1.1–2.0×

*factor = tiempo_nodo_único / tiempo_clúster (>1 significa que el clúster fue más rápido). Rangos observados entre dos corridas independientes de 5 mediciones cada una; ver `evidencia/resultados.csv` para los datos crudos.

Interpretación. El patrón es consistente con la teoría del Capítulo 2: en Q1 (lectura puntual por clave primaria), el nodo único gana porque no hay trabajo que paralelizar — el único costo es la coordinación del quórum Raft, que el nodo único no paga. En Q4 y Q5, las consultas con más trabajo de cómputo (agregación sobre 20 000 filas y *join*), el plan distribuido del clúster (`nodes: n1, n2` en el `EXPLAIN ANALYZE`) sí aporta paralelismo real y el clúster gana con margen consistente. Q2 y Q3 quedan en un punto intermedio donde el resultado depende de si la consulta toca una sola partición (Q2, favorece al clúster por localidad de partición) o cruza las tres (Q3, cercano al empate). Esto confirma que, a este volumen de datos (20 000 filas), la ventaja de la distribución no es universal sino que depende del tipo de consulta — observación que sería clave para dimensionar cuándo migrar a un clúster de más de un nodo en un escenario de producción real del PFC ACC.

5. Preguntas de análisis obligatorias

1. ¿Cómo garantiza CockroachDB consistencia ante la caída de un nodo?

CockroachDB divide el espacio de claves en rangos (*ranges*) y replica cada uno en un grupo Raft independiente, típicamente de tres réplicas [4]. Dentro de cada grupo, Raft elige un líder por mayoría; solo el líder acepta escrituras nuevas, las anexa a su log replicado y las propaga a los seguidores [3]. Una escritura se confirma únicamente cuando la mayoría de las réplicas del rango ha persistido esa entrada del log (en un grupo de tres, dos de tres), lo que impide que dos líderes comprometan simultáneamente entradas conflictivas para la misma posición del log [3]. En la prueba de la Sección 3, al detener `crdb-node2`, cada rango en el que este participaba perdió una réplica, pero mientras el nodo caído no fuera el líder de un rango dado, ese rango siguió respondiendo gracias al quórum de 2/3 restante. Si hubiera sido líder, los seguidores habrían detectado la ausencia de *heartbeats* y disparado una nueva elección en cuestión de segundos. Ningún dato se pierde porque toda escritura confirmada ya estaba persistida en al menos dos réplicas antes de reportarse exitosa al cliente; al reintegrarse el nodo (~65s en la prueba), simplemente recibe del líder las entradas de log que le faltan, sin intervención manual [3], [4].

2. CockroachDB (serializable) vs. Cassandra (consistencia eventual): ¿cuál para el PFC ACC?

Para el dominio de ACC, la elección correcta es CockroachDB, y el modelo PACELC de Abadi explica por qué [23]. Cassandra prioriza disponibilidad y baja latencia incluso a costa de consistencia, con un modelo eventual ajustable por nivel de quórum de lectura y escritura [25], adecuado para casos donde una lectura ligeramente obsoleta es tolerable, como un contador de *likes*. Pero en ACC la operación crítica es la asignación de un ticket a un técnico: bajo consistencia eventual, dos técnicos podrían leer simultáneamente el mismo ticket como disponible y ambos intentar tomarlo, generando doble trabajo o dejando sin asignar un ticket con SLA crítico. CockroachDB evita esto por diseño, al implementar aislamiento serializable estricto, apoyado en *timestamps* de reloj híbrido y en el orden total que impone Raft dentro de cada rango [4]. En términos de PACELC, esto significa que incluso sin partición de red (rama *Else*), CockroachDB elige sistemáticamente consistencia sobre latencia: acepta una escritura algo más lenta (el costo del *round-trip* del consenso Raft) a cambio de que **ticket-service** nunca necesite lógica de resolución de conflictos en el cliente, algo que sí sería obligatorio con Cassandra. Dado que el volumen de escrituras de un ISP regional es modesto frente a la escala geo-distribuida para la que Cassandra fue pensada, ese costo de latencia adicional es aceptable y la ganancia en simplicidad y corrección lo justifica ampliamente [14].

3. Diseño de la fragmentación horizontal explícita para tickets

La estrategia implementada usa `PARTITION BY LIST` sobre la columna **zone**, con las particiones `tickets_centro`, `tickets_norte`, `tickets_sur` y una partición `DEFAULT` de resguardo para valores no anticipados. Se eligió `LIST` y no `RANGE` porque **zone** es un conjunto pequeño, fijo y discreto de tres categorías sin relación de orden entre ellas; `RANGE` tiene sentido para claves continuas y ordenables, por ejemplo particionar por la fecha de creación para descartar datos antiguos, pero forzar un orden artificial sobre zonas geográficas no relacionadas habría sido semánticamente incorrecto [9], [10]. La columna **zone** se incluyó además en la clave primaria compuesta junto al identificador, un requisito técnico de CockroachDB para que cada fila pertenezca inequívocamente a una sola partición sin tener que reevaluar el predicado en cada operación. La justificación de negocio es que el patrón de acceso dominante de **ticket-service** (un técnico consultando los tickets de su propia zona) alinea el filtro más común con la clave de partición, lo que, como se observó en la consulta Q2 de la Sección 4, permite resolver la consulta tocando un solo rango en vez de dispersar la lectura entre las tres particiones. Cada partición además se ancla mediante `CONFIGURE ZONE` con una restricción de localidad hacia el nodo de su misma zona, acercando el cómputo a donde probablemente se originan las lecturas más frecuentes, sin renunciar a las tres réplicas completas que exige la tolerancia a fallos del clúster [4].

4. Atomicidad de transacciones multi-tabla en distintos nodos

Cuando una transacción de ACC modifica átomicamente filas en rangos distintos (por ejemplo, el estado de un ticket y un contador de carga de un técnico) CockroachDB no depende de un protocolo de *commit* en dos fases clásico entre nodos independientes, sino que compone la atomicidad a partir de Raft y de un protocolo transaccional propio [4]. Uno de los rangos involucrados se designa como registro transaccional, donde se anota el estado de la transacción como pendiente, confirmada o abortada. Cada escritura individual se propone primero como un valor provisional o *intent*, replicado con las garantías normales de Raft dentro del grupo de su propio rango, de forma independiente y en paralelo con los demás rangos afectados [3], [4]. Solo cuando todos los *intents* han sido comprometidos por el quórum Raft de su rango correspondiente, el coordinador marca el registro transaccional como confirmado; los *intents* se resuelven después de forma asíncrona, y cualquier lector que los encuentre antes puede consultar ese registro para saber si debe considerarlos válidos. Si el coordinador falla a mitad de camino, cualquier otro

nodo que lea un *intent* puede, tras un tiempo de espera, resolver la transacción consultando el registro, evitando el bloqueo indefinido característico del *commit* en dos fases clásico [19]. En suma, la atomicidad multi-tabla se logra componiendo varias instancias de consenso Raft bajo un único registro transaccional como fuente de verdad, no mediante un coordinador central que deba permanecer vivo [4].

6. Análisis crítico

La verificación experimental confirmó lo que predice la teoría de la Sección 2: el clúster resistió la caída de un nodo sin dejar de funcionar y sin perder información, tal como se espera del algoritmo Raft cuando se mantiene la mayoría de los nodos activos, es decir, un quórum de 2 de 3 [3]. Sin embargo, al medir el rendimiento apareció un matiz que la teoría por sí sola no deja ver con claridad: con 20 000 filas, el clúster distribuido no siempre fue más rápido que una sola instancia de base de datos, sino que esto dependió del tipo de consulta que se ejecutara. Esto coincide con lo que plantea Abadi sobre que todo sistema distribuido paga un costo de tiempo por mantener la información consistente entre nodos, incluso cuando no hay ningún problema de red de por medio [23]: ese costo, que implica que los nodos se pongan de acuerdo entre sí antes de confirmar cada operación, se nota más en consultas simples (como buscar un ticket por su identificador) y se compensa solo cuando la consulta requiere suficiente trabajo como para que repartirlo entre varios nodos realmente ayude. Una limitación real de esta práctica es que se realizó en un entorno de desarrollo local (contenedores Docker en una sola máquina), donde los tres nodos comparten el mismo procesador, disco y red que el propio cliente que hace las consultas; en un despliegue real con los nodos ubicados en distintas regiones (el escenario para el que CockroachDB fue pensado) es de esperar que la ventaja de distribuir los datos sea mayor en consultas que aprovechan la localidad de la partición (Q2), y que el costo de mantener consistencia se sienta más en el tiempo que toma la comunicación entre regiones que en el procesamiento local. Para el PFC ACC, la recomendación práctica que se desprende de este análisis es priorizar el diseño de índices y particiones alineados con los patrones de consulta reales del sistema (como ya se hizo con *zone*), en lugar de asumir que "más nodos" es automáticamente "más rápido" para cualquier tipo de consulta.

7. Conclusiones

Se implementó y verificó experimentalmente un clúster CockroachDB de tres nodos aplicado al esquema del PFC ACC, demostrando fragmentación horizontal por zona con las tres condiciones de corrección satisfechas, tolerancia a fallos por quórum Raft sin pérdida de datos ante la caída de un nodo, y una ganancia de rendimiento distribuido que es real pero dependiente del tipo de consulta, no universal. El ejercicio confirma que la teoría de bases de datos distribuidas (fragmentación, replicación, consenso y los trade-offs de PACELC) no es solo material de examen, sino que se puede observar directamente en el comportamiento medible de un sistema real.

Conclusiones individuales

Jhinson Stalyn Aucatoma Celorio: En esta práctica se pudo comprobar que la fragmentación de datos, un concepto que en el fundamento teórico se presenta como un objetivo de diseño, realmente funciona en un sistema como CockroachDB sin complicar el uso de la base de datos. Al dividir la tabla de tickets por zona (centro, norte y sur de Quevedo), se pudo verificar que cada partición se guardaba correctamente en el clúster y que las consultas seguían funcionando igual que si la información estuviera en un solo lugar. Este proceso confirma que la teoría sobre fragmentación tiene una aplicación concreta y medible. Además, se pudo evidenciar que no fue necesario crear una base de datos nueva para el PFC ACC, sino que se reutilizó el esquema ya

existente del sistema de tickets, adaptándolo al entorno distribuido. Esto demuestra que estos conceptos pueden aplicarse de forma progresiva sobre un sistema ya construido, sin tener que rediseñarlo por completo.

Jeremy Alexis Alvarez Parraga: Con la prueba de tolerancia a fallos llevada a cabo de manera práctica pudo mostrar algo que antes solo se veía de forma teórica: cuando uno de los tres nodos del clúster se detiene, el sistema sigue funcionando siempre que los otros dos continúen de acuerdo entre sí, es decir, mientras se mantenga la mayoría o "quórum". Al detener el nodo `crdb-node2`, las consultas siguieron respondiendo correctamente y no se perdió información, lo cual respalda directamente lo estudiado sobre el algoritmo Raft. También se observó que, después de 30 segundos con el nodo caído, el sistema no reorganizó de inmediato la información entre los nodos restantes, y esto tiene una explicación sencilla: el clúster espera varios minutos antes de asumir que un nodo está definitivamente fuera de servicio, para no reaccionar de más ante fallas breves o temporales. Cuando el nodo se reinició, se puso al día con la información faltante en menos de un minuto, sin necesidad de elegir un nuevo líder. Todo esto demuestra que un sistema como el de ACC podría mantenerse disponible incluso ante fallas de hardware o de red.

Carlos Jose Carpio Mendoza: Al comparar el rendimiento del clúster distribuido frente a una sola instancia de base de datos, se pudo descubrir que tener más nodos no siempre significa mayor velocidad. En la consulta más simple, buscar un ticket por su identificador, la base de datos de un solo nodo fue más rápida, porque no hay ninguna tarea que se pueda repartir entre varios nodos y, en cambio, el clúster sí debe invertir tiempo en que los nodos se pongan de acuerdo entre sí. En cambio, en consultas más exigentes, como calcular estadísticas por zona y categoría, o relacionar tickets con técnicos, el clúster resultó ser claramente más rápido, porque ahí sí puede repartir el trabajo entre varios nodos al mismo tiempo. Esto confirma la idea de que mantener la información siempre consistente entre nodos tiene un costo, y que ese costo solo vale la pena cuando la consulta implica suficiente trabajo como para aprovechar la distribución. Para el PFC ACC, esto sugiere que antes de adoptar un clúster distribuido conviene analizar bien qué tipo de consultas se van a ejecutar con más frecuencia.

Declaración de uso de inteligencia artificial generativa

En la elaboración de este informe se utilizó un asistente de inteligencia artificial generativa (Claude, de Anthropic) como herramienta de apoyo, no como autor del contenido técnico. Su uso se limitó a tres propósitos concretos. Primero, la verificación de las referencias bibliográficas citadas en el fundamento teórico, confirmando que los DOI e ISBN correspondieran efectivamente a la fuente citada y que las editoriales fueran de origen académico reconocido, descartando fuentes depredadoras o no arbitradas. Segundo, el apoyo en la redacción del Análisis crítico (Sección 6), las Preguntas de análisis obligatorias (Sección 5) y las conclusiones individuales (Sección 7): en los tres casos, las ideas, los hallazgos y las conclusiones fueron redactados originalmente por los integrantes del equipo a partir de su propia experiencia con el desarrollo de la práctica y de la evidencia experimental obtenida; la herramienta se empleó únicamente para mejorar la coherencia, la fluidez y la conexión entre oraciones del texto ya elaborado por los integrantes, sin introducir afirmaciones, datos o interpretaciones nuevas. Tercero, se usó de forma puntual para revisar la consistencia gramatical general del documento. En ningún momento la IA generó por sí sola resultados experimentales, código de implementación ni las cifras del análisis comparativo de rendimiento.

Referencias

- [1] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [2] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.^a ed. Cham, Switzerland: Springer, 2020, ISBN: 978-3-030-26252-5. DOI: 10.1007/978-3-030-26253-2.
- [3] D. Ongaro y J. Ousterhout, «In Search of an Understandable Consensus Algorithm,» en *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, Philadelphia, PA, USA: USENIX Association, 2014, págs. 305-319, ISBN: 978-1-931971-10-2.
- [4] R. Taft, I. Sharif, A. Matei et al., «CockroachDB: The Resilient Geo-Distributed SQL Database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, Portland, OR, USA: ACM, 2020, págs. 1493-1509. DOI: 10.1145/3318464.3386134.
- [5] C. J. Date, *Introducción a los Sistemas de Bases de Datos*, 7.^a ed. México: Pearson Educación, 2001, ISBN: 968-444-419-2.
- [6] G. Coulouris, J. Dollimore, T. Kindberg y G. Blair, *Distributed Systems: Concepts and Design*, 5.^a ed. Addison-Wesley, 2012, ISBN: 978-0-13-214301-1.
- [7] P. Valduriez, «Principles of Distributed Data Management in 2020?» En *Database and Expert Systems Applications (DEXA 2011)*, Toulouse, France: Springer, 2011. DOI: 10.1007/978-3-642-23088-2_1.
- [8] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 3.^a ed. distributed-systems.net, 2017, ISBN: 978-1543057386.
- [9] S. Mehta, P. Agarwal, P. Shrivastava y J. Barlawala, «Differential Bond Energy Algorithm for Optimal Vertical Fragmentation of Distributed Databases,» *Journal of King Saud University – Computer and Information Sciences*, vol. 34, n.º 1, págs. 1466-1471, 2022. DOI: 10.1016/j.jksuci.2018.09.020.
- [10] Y.-F. Ge et al., «Evolutionary Dynamic Database Partitioning Optimization for Privacy and Utility,» *IEEE Transactions on Dependable and Secure Computing*, vol. 21, n.º 4, págs. 2296-2311, 2024. DOI: 10.1109/TDSC.2023.3302284.
- [11] D. Nashat y A. A. Amer, «A Comprehensive Taxonomy of Fragmentation and Allocation Techniques in Distributed Database Design,» *ACM Computing Surveys*, vol. 51, n.º 1, págs. 1-25, 2018. DOI: 10.1145/3150223.
- [12] D. Rambabu y A. Govardhan, «Survey on Data Replication in Cloud Systems,» *Web Intelligence*, vol. 22, n.º 1, págs. 83-109, 2024. DOI: 10.3233/WEB-230087.
- [13] R. Mokadem et al., «A Review on Data Replication Strategies in Cloud Systems,» *International Journal of Grid and Utility Computing*, vol. 13, n.º 4, págs. 347-362, 2022. DOI: 10.1504/IJGUC.2022.125135.
- [14] P. Viotti y M. Vukolić, «Consistency in Non-Transactional Distributed Storage Systems,» *ACM Computing Surveys*, vol. 49, n.º 1, págs. 1-34, 2016. DOI: 10.1145/2926965.
- [15] J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko y A. Sheikh-Akbari, «Consistency Issue and Related Trade-offs in Distributed Replicated Systems and Databases: A Review,» *Radioelectronic and Computer Systems*, vol. 2, n.º 106, págs. 171-179, 2023. DOI: 10.32620/reks.2023.2.14.
- [16] M. Hasan y S. Yasmin, «Concurrency Control in Distributed Database System: Solution of the Anomalies & Challenges,» en *Proceedings of the International Conference on Computing Advancements*, 2025. DOI: 10.1145/3723178.3723261.

- [17] O. Bukhres y K. Magel, «Deadlock Detection in Distributed Database Systems: A Performance Evaluation Study,» en *Proceedings of the Fifteenth Annual International Computer Software and Applications Conference (COMPSAC '91)*, 1991, págs. 78-83. DOI: 10.1109/COMPSAC.1991.170155.
- [18] P. Fan, J. Liu, W. Yin, H. Wang, X. Chen y H. Sun, «2PC*: A Distributed Transaction Concurrency Control Protocol of Multi-Microservice Based on Cloud Computing Platform,» *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 9, n.º 40, 2020. DOI: 10.1186/s13677-020-00183-w.
- [19] D. Skeen, «Nonblocking Commit Protocols,» en *Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data*, Ann Arbor, MI, USA: ACM, 1981, págs. 133-142. DOI: 10.1145/582318.582339.
- [20] N. Kumar, L. Sahoo y A. Kumar, «Design and Implementation of Three Phase Commit Protocol (3PC) Algorithm,» en *2014 International Conference on Reliability, Optimization and Information Technology (ICROIT)*, India, 2014, págs. 116-120. DOI: 10.1109/ICROIT.2014.6798309.
- [21] S. Gilbert y N. Lynch, «Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: 10.1145/564585.564601.
- [22] E. Brewer, «CAP Twelve Years Later: How the "Rules" Have Changed,» *Computer*, vol. 45, n.º 2, págs. 23-29, 2012. DOI: 10.1109/MC.2012.37.
- [23] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: 10.1109/MC.2012.33.
- [24] G. DeCandia, D. Hastorun, M. Jampani et al., «Dynamo: Amazon's Highly Available Key-value Store,» en *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*, Stevenson, WA, USA: ACM, 2007, págs. 205-220. DOI: 10.1145/1294261.1294281.
- [25] A. Lakshman y P. Malik, «Cassandra: A Decentralized Structured Storage System,» *ACM SIGOPS Operating Systems Review*, vol. 44, n.º 2, págs. 35-40, 2010. DOI: 10.1145/1773912.1773922.
- [26] W. Schultz, T. Avitabile y A. Cabral, «Tunable Consistency in MongoDB,» *Proceedings of the VLDB Endowment*, vol. 12, n.º 12, págs. 2071-2081, 2019. DOI: 10.14778/3352063.3352125.
- [27] J. C. Corbett, J. Dean, M. Epstein et al., «Spanner: Google's Globally Distributed Database,» *ACM Transactions on Computer Systems*, vol. 31, n.º 3, págs. 1-22, 2013. DOI: 10.1145/2491245.